WEB APPLICATION PENETRATION TEST — VULNERABILITY REPORT

Stored Cross-Site Scripting — Post Body Rendered Unencoded

Finding NVP-006 · **High**

Product: PHPGurukul News Portal Project in PHP and MySQL — V4.1

Assessment target: http://localhost/newsportal/newsportal/

Technology: Apache 2.4.58 · PHP 8.2.12 · MariaDB 10.4.32 (DB user: root)

Assessment date: 2026-10-31

Classification: CONFIDENTIAL — authorised lab assessment (review draft)

Document Control

Finding ID	NVP-006
Vulnerability	Stored Cross-Site Scripting — Post Body Rendered Unencoded
Severity	**High**
CVSS 4.0	7.0 `CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:P/VC:L/VI:L/VA:L/SC:H/SI:H/SA:N`
CWE	CWE-79 (Cross-site Scripting)
OWASP	A03:2021 Injection
CVE verification	No matching publicly assigned CVE was identified for this specific finding and product/version during the research performed.
Status	CONFIRMED (manually validated & re-verified)

1. Executive Summary

The post body is stored and then rendered on the public news-detail page without output encoding, so markup/JavaScript placed in a post executes in every visitor's browser.

2. Affected Target

Affected URL	Stored via /admin/add-post.php (or edit-post.php); executes on /news-details.php
Affected parameter	postdescription (POST)
Affected role	Author: admin/sub-admin · Victim: any public visitor
Authentication	Authoring requires a login; execution affects unauthenticated visitors

3. Technical Description

news-details.php renders the post body with echo substr($pt,0) — no htmlentities/htmlspecialchars. add-post.php applies only addslashes (SQL-safe, not HTML-safe) and edit-post.php applies no encoding. A post whose body contains a script or an onerror handler therefore runs as active content for all readers. (Post titles ARE encoded elsewhere, so the body is the exploitable sink.)

4. Root Cause Analysis

```
news-details.php:119-121
$pt=$row['postdetails'];
echo (substr($pt,0));   // printed raw, no HTML encoding
// stored by admin/add-post.php (addslashes only) and admin/edit-post.php (no encoding).
```

5. Preconditions

Ability to create/edit a post (admin or sub-admin — and note sub-admins can escalate via NVP-005). Victims only need to view the article.

6. Proof of Concept — Step-by-Step Reproduction

1. Authenticate to the admin panel and create a post (title "VAPT StoredXSS Demo b7").
2. Set the post body to a benign marker payload: <script>window.__vaptStored=1;document.title="XSS-FIRED"</script><img src=x onerror="document.title='XSS-FIRED-IMG'">VAPT-STORED-XSS-b7
3. Open the public article /news-details.php?nid=14 as an unauthenticated visitor.
4. The script executes: window.__vaptStored===true and document.title becomes "XSS-FIRED-IMG"; the renders as markup, the <img> fires onerror. (Harmless markers used instead of a real payload.)

7. HTTP Request Evidence

```
GET /newsportal/newsportal/news-details.php?nid=14 HTTP/1.1
Host: localhost
```

8. HTTP Response Evidence

```
HTTP/1.1 200 OK
...
<p class="card-text"><script>window.__vaptStored=1;document.title="XSS-FIRED"</script><img src=x
onerror="..."><b>VAPT-STORED-XSS-b7</b></p>  <!-- served unencoded -->
```

9. Proof of Concept — Visual Evidence

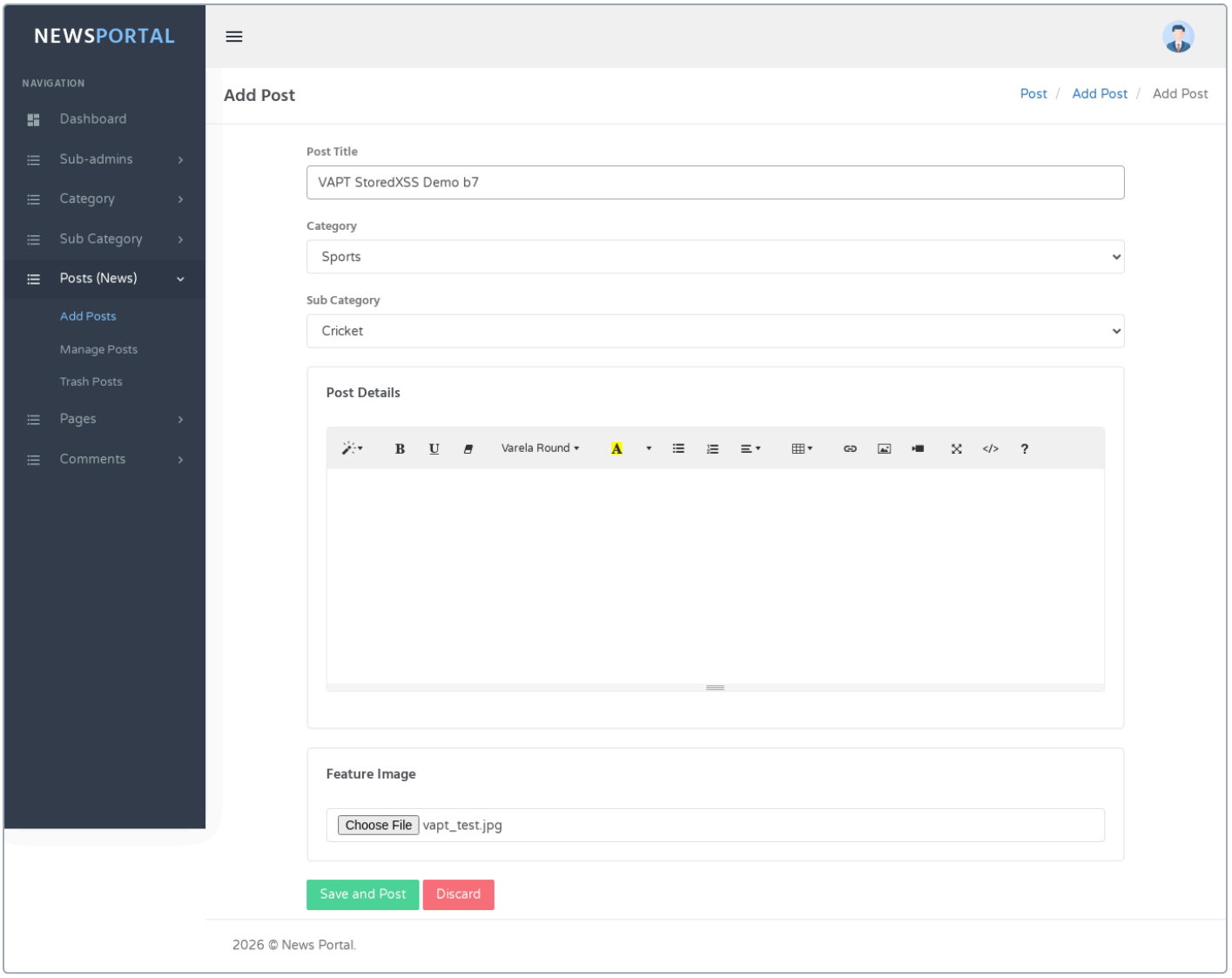


Figure 1 — Benign XSS marker entered as the post body in Add Post.

> What this proves: Shows the authoring injection point (postdescription).

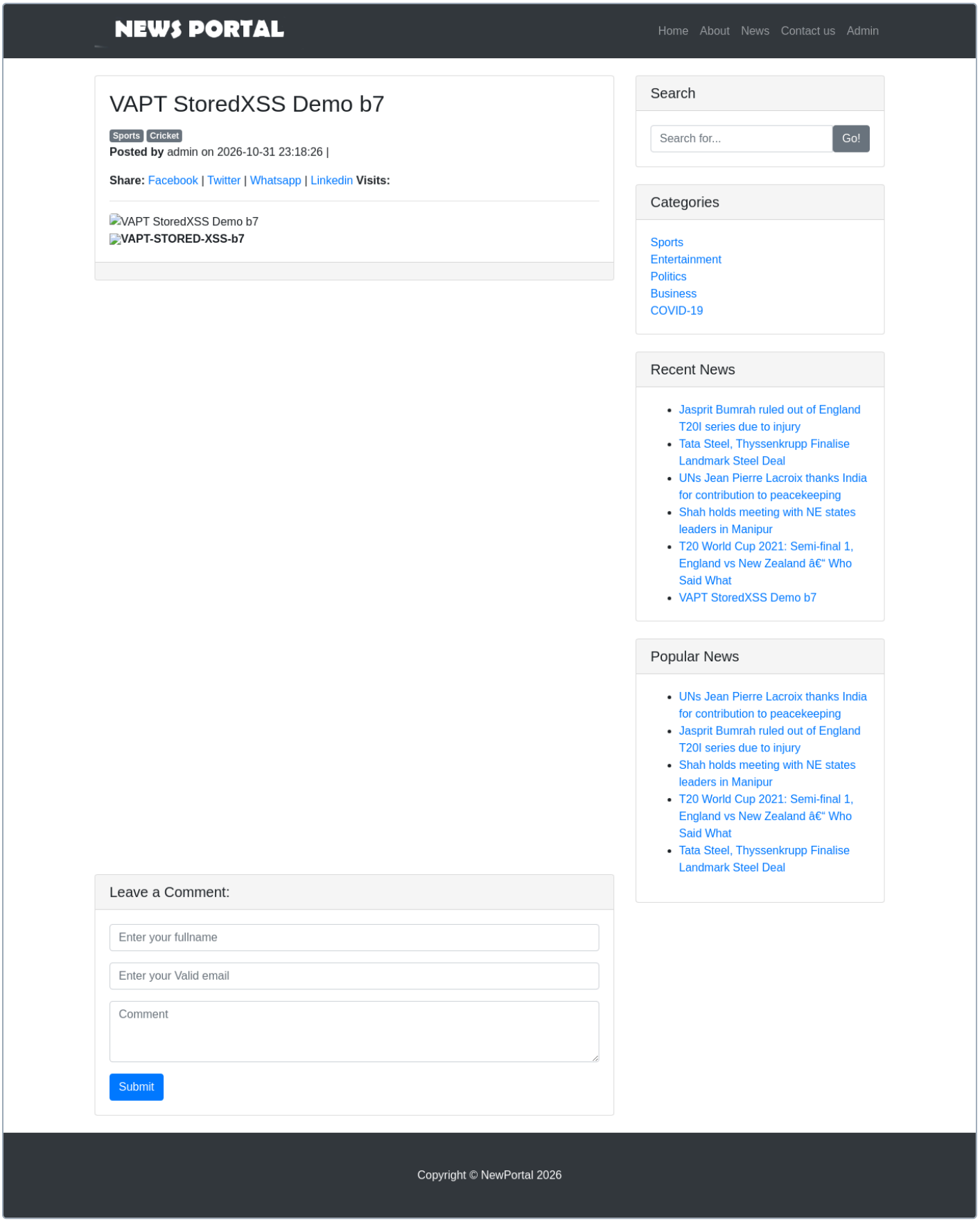


Figure 2 — Public article renders the payload as live HTML/JS (broken <img>, bold marker; title set to "XSS-FIRED-IMG").

> What this proves: Injected script/markup executes for an unauthenticated visitor (window.__vaptStored===true verified), confirming stored XSS.

10. Actual Security Impact

Persistent client-side code execution against every visitor of an affected article: session/cookie theft is limited by cookie flags but admin-session hijacking, credential phishing, drive-by redirects, and site defacement are all feasible. Combined with NVP-005 even a sub-admin can plant it.

Exploitation complexity	Low to store; victims require only to view the page (passive interaction).
Realistic attack scenario	A sub-admin (or attacker who gained admin via NVP-001/004) publishes a booby-trapped article; every reader's browser runs attacker JavaScript.

11. Remediation Recommendations

HTML-encode all stored content on output (htmlspecialchars with ENT_QUOTES) and/or sanitise the rich-text with an allowlist (e.g. HTML Purifier). Add a Content-Security-Policy. Do not rely on addslashes for HTML safety.

Secure code example

```
echo htmlspecialchars($row['postdetails'], ENT_QUOTES, 'UTF-8');
// or render through a vetted HTML sanitiser allowlist for legitimate rich text.
```

12. Retesting Procedure

Re-open the article; a fixed system displays the payload as inert text and does not execute it.

13. References

- CWE-79
- OWASP WSTG-INPV-02
- OWASP A03:2021

14. Conclusion

Confirmed stored XSS executing on the public site. Verified via real admin authoring + unauthenticated execution (document.title / JS variable changed).